The `require_tree` directive tells Sprockets to recursively include *all* JavaScript files in the specified directory into the output. These paths must be specified relative to the manifest file. You can also use the `require_directory` directive which includes all JavaScript files only in the directory specified, without recursion.
Directives are processed top to bottom, but the order in which files are included by `require_tree` is unspecified. You should not rely on any particular order among those. If you need to ensure some particular JavaScript ends up above some other in the concatenated file, require the prerequisite file first in the manifest. Note that the family of `require` directives prevents files from being included twice in the output.
Rails also creates a default `app/assets/stylesheets/application.css` file which contains these lines:

```
/* ...
*= require_self
*= require_tree .
*/
```

Rails 4 creates
both `app/assets/javascripts/application.js` and `app/assets/stylesheets/application.css` re gardless of whether the --skip-sprockets option is used when creating a new rails application. This is so you can easily add asset pipelining later if you like.
The directives that work in JavaScript files also work in stylesheets (though obviously including stylesheets rather than JavaScript files). The `require_tree` directive in a CSS manifest works the same way as the JavaScript one, requiring all stylesheets from the current directory.
In this example, `require_self` is used. This puts the CSS contained within the file (if any) at the precise location of the `require_self` call.
If you want to use multiple Sass files, you should generally use the Sass `@import` ruleinstead of these Sprockets directives. When using Sprockets directives, Sass files exist within their own scope, making variables or mixins only available within the document they were defined in.

You can do file globbing as well using `@import "*"`, and `@import "**/*"` to add the whole tree which is equivalent to how `require_tree` works. Check the sass-rails documentation for more info and important caveats.
You can have as many manifest files as you need. For example, the `admin.css` and `admin.js`manifest could contain the JS and CSS files that are used for the admin section of an application.
The same remarks about ordering made above apply. In particular, you can specify individual files and they are compiled in the order specified. For example, you might concatenate three CSS files together this way:

```
/* ...
*= require reset
*= require layout
*= require chrome
*/
```

2.5 Preprocessing

The file extensions used on an asset determine what preprocessing is applied. When a controller or a scaffold is generated with the default Rails gemset, a CoffeeScript file and a SCSS file are generated in place of a regular JavaScript and CSS file. The example used before was a controller called "projects", which generated an `app/assets/javascripts/projects.coffee` and
an `app/assets/stylesheets/projects.scss` file.
In development mode, or if the asset pipeline is disabled, when these files are requested they are processed by the processors provided by the `coffee-script` and `sass` gems and then sent back to the